2

计算复杂性理论基础

计算复杂性理论（computational complexity theory）研究的是可计算问题根据它们本身的复杂性进行的分类，以及将这些类别联系起来的方法。一个可计算问题被认为是一个原则上可以用计算机解决的问题，即这个问题可以用一系列机械的数学步骤（如算法）来解决。计算复杂性理论主要关注算法在执行时所需的计算资源，特别是时间和空间资源。

注意，计算复杂性理论和可计算性理论（computability theory）是不同的：

- **可计算性理论**研究的核心问题是：这个问题是否可以被算法解决？不可解问题是该领域的核心发现。
- **计算复杂性理论**研究的核心问题是：如果这个问题可解，解决它需要多少资源（如时间、空间）？即如何高效解决可解问题，关注算法的效率，其核心分类包括 P 问题、NP 问题、NPH 问题、NPC 问题等。

通俗来说，可计算性理论研究“存在性”问题：能不能算？而计算复杂性理论研究“效率性”问题：算得多快、多省？两者共同构建了计算机科学的理论基础，前者划定边界，后者指引实践。

此外，对一个有明确定义的算法的复杂度进行的研究，叫做**算法分析**（analysis of algorithm），而对可解问题的复杂度进行的研究，叫做**计算复杂性理论分析**。下面我们首先介绍复杂度，然后介绍 P 问题、NP 问题、NPH 问题、NPC 问题的概念。

MPC 对计算复杂性理论有着根本性需求，因为其安全性定义（如计算不可区分性）和可行性证明都强烈依赖于“计算困难性”这一核心概念，也就是说，MPC 协议的安全性通常建立在对手的计算能力有限（通常是概率多项式时间）这一假设之上，它需要复

杂性理论来严格界定哪些问题在计算上是“难解”的，从而确保即使对手看到协议交互信息也无法推算出超出预设的私密信息。

另外请注意，计算复杂性理论与密码学之间是奠基与依存的核心关系：密码学的整个安全体系建立在计算复杂性理论对“计算困难性”的严格定义和分类之上。具体而言，密码学协议（如加密算法、数字签名等）的安全性依赖于“存在在标准计算模型下难以求解的数学问题”这一复杂性理论假设（例如，大整数分解或离散对数问题不属于易解的 P 类）。复杂性理论提供了衡量攻击者（敌手）$\mathcal{A}$ 计算能力的框架（如多项式时间、概率多项式时间），使得密码学能够精确定义“安全”意味着攻击者 $\mathcal{A}$ 即使拥有多项式时间计算资源，其成功概率也可忽略不计。同时，密码学的实践需求（如伪随机数生成、零知识证明）也反过来推动和丰富了复杂性理论的研究，催生了如平均情况复杂性、密码学复杂性类等新方向。因此，复杂性理论为密码学提供了不可或缺的理论基石和安全性证明的语言，而密码学则是复杂性理论最深刻和成功的应用领域之一。正因如此，本书中首先介绍计算复杂性理论。

2.1 具体复杂度和渐进复杂度

复杂度（complexity）是一个相对广泛的概念，可以包括**具体复杂度**（concrete complexity）和**渐进复杂度**（asymptotic complexity）两个视角。

首先看具体复杂度。具体复杂度用来表征算法在具体输入下的精确资源消耗（如时间、空间），用数学表达式或具体数值来表示，保留低阶项并明确包含常数因子，比如 $T(n) = 3n^2 + 2n + 300$。而渐进复杂度是指算法在输入规模趋于无穷大时的复杂度。换句话说，它表征算法在运行时所消耗时间的增长速率，随着输入规模的增加，这种增长速率也会不断增加。在算法分析中，渐进复杂度是一种重要的指标，包括时间复杂度和空间复杂度等，用于评估算法的效率和可行性。例如，如果一个算法的时间复杂度为 $O(n^2)$，这意味着当输入规模 n 增加时，算法运行时间将按输入规模的平方的速度增长。显然，该算法在处理大量数据时将非常耗时，可能需要考虑改进算法或寻找更高效的算法来提高效率。

之所以需要渐进复杂度，主要是为了简化分析，因为实际算法的具体复杂度可能非常复杂（比如 $T(n) = 0.5n^2 + 20n + 3000$），而渐进复杂度（比如 $O(n^2)$）能快速揭示其本质。

2.2 计算复杂度和通信复杂度

计算复杂度和通信复杂度是两个在计算机科学领域中用于衡量算法或协议性能的重要指标。

2.2.1 计算复杂度

在计算机科学（computer science）中，一个算法（algorithm）的计算复杂度就是运行这个算法所需要的资源量，特别是时间（占用 CPU 的时间）和空间（占用内存的空间大小）需求。因此，计算复杂度包括时间复杂度（time complexity）和空间复杂度（space complexity）。

时间复杂度通常表示为对一个输入值长度所需基本操作（通常是加法操作或者乘法操作）的数量。我们假设基本操作在一台计算机上只占用一个不变的时间量（比如 1 纳秒），而在另一台计算机上运行时，只根据一个常量因子进行改变（比如 $k * 1$ 纳秒）。空间复杂度通常表示为算法对一个输入值长度所需的内存量。在计算机科学和算法分析中，除非特别指出，当提到计算复杂度时，通常是指时间复杂度，因为时间复杂度是衡量算法效率的关键因素，直接关系到算法在实际应用中的表现。

2.2.1.1 复杂度的表示方法

由于运行一个算法所需的资源量通常随输入规模 n 的大小而变化，因此复杂度（包括时间复杂度和空间复杂度）通常用函数 $f(n)$ 表示，$f(n)$ 是最坏情况复杂度（worst-case complexity），即输入大小为 n 时所需的资源量的最大值，或是平均情况复杂度（average-case complexity），即资源总量相对于 n 的所有占用的平均值。举例来说，怎样把一串数字从小到大进行排序，是一个问题；而针对这一问题，我们有多种明确定义的算法，如选择排序，归并排序等。假设我们有 n 个数字，那么排序问题的复杂度就是 $n\log(n)$，而选择排序的复杂度是 n^2，归并排序的复杂度是 $6n\log(n)$。

2.2.1.2 复杂度的评估模型

另外，复杂度的评估依赖于选取的计算模型（model of computation），计算模型可分为确定性计算模型和非确定性计算模型两种。当计算模型没有被明确指定时，通常假设使用的是确定性计算模型，尤其是多带图灵机（multitape Turing machine）。

- **确定性计算模型**（deterministic model of computation）是一种在计算过程中，机器的后续状态和要执行的操作完全由前面的状态决定的计算模型；

- **非确定性计算模型**（non-deterministic model of computation）是指在计算的某些步骤中，可能会进行一些选择的计算模型。在复杂性理论中，非确定性时间复杂度即为：同时考虑所有可能的选择且做出最佳选择时所需的时间。换句话说，计算被认为是在多个（相同的）处理器上同时进行的，而不确定计算时间是第一个完成计算的处理器所花费的时间。

2.2.1.3 多项式时间和概率多项式时间

一个问题的时间复杂度与**多项式时间**（polynomial time）的概念相关。多项式时间是指一个问题的计算时间（即算法的运行时间）不大于问题规模 n 的多项式倍数，如 $O(n)$、$O(n^2)$、$O(\log(n))$ 等。通常要求要研究的问题存在多项式时间算法，这是因为：研究一个时间复杂度比多项式算法还要复杂的算法是没有实际意义的。

时间复杂度是指随着问题规模（输入资料的大小）n 的增长，求解该问题所花的时间的增长情况，比如：

- $O(1)$ 表示随着问题规模的增长，求解该问题所花的时间不变，即与问题的规模无关；
- $O(n)$ 表示随着问题规模的增长，所花的时间和问题规模成正比。

此外，还有 $O(n^2)$、$O(\log(n))$、$O(a^n)$、$O(n!)$ 等。因此，在计算复杂度理论中，多项式时间是指求解一个问题所花的时间 $t(n)$ 不大于问题规模 n 的多项式倍数，即可用关于问题规模的多项式来衡量求解问题所需的时间。使用多项式时间这一概念，主要是因为虽然所花的时间随着 n 的增大而快速增大，但这种增长趋势是可被计算机接受的，而像指数级增长和阶乘级增长这种增长趋势则是不可接受的。

还有一个相关的概念是**概率多项式时间**（Probability Polynomial Time，PPT）。为便于理解这个概念，这里以概率多项式时间算法（PPT 算法）为例，其中的“概率”用以说明在算法执行过程中，存在随机选择下一步的过程（即该算法为概率算法），也就是说，对于相同的输入，经过 PPT 算法处理后，可能得到不同的结果（求解结果具有概率性，但结果的偏差有界），往往需要多次尝试求解，即：“概率”一词引入了随机性，而随机性对密码学来说是非常重要的。与 PPT 相对应的概念是确定性多项式时间，其中的“确定性”是指两次或多次求解同一个问题，得到的结果是相同的。

另外，有很多算法的计算时间比多项式时间慢，因此被称为超多项式时间，但又比指数时间快，因此又被称为次指数时间。

2.2.2 通信复杂度

通信复杂度是指在分布式系统中，计算节点之间进行通信时所需的通信传输开销。比如在联邦学习模型训练过程中，计算节点需要交换参数和梯度等信息以进行模型更新，会带来较大的通信开销。通信复杂度的增加主要与网络带宽和通信量有关，通信量的增加会使通信过程变得更为复杂。

通信复杂度通常研究的是如何发送尽可能少的比特数（或域元素，field elements，实际上是指域中元素的位长，即 bit length），从而更高效地得到计算结果。度量通信复杂度有两种方式：一是**轮复杂度**（round complexity），是指执行协议时需要的通信次数或轮次；二是**位复杂度**，指协议执行中由诚实方发送和接收的总比特数（total number of bits），也就是信息传输量。

2.3 P 问题、NP 问题、NPH 问题和 NPC 问题

P 问题、NP 问题、NPH 问题和 NPC 问题是算法领域及计算复杂性理论中的四个重要问题，主要用来衡量问题的“难易”程度，即一个问题在计算资源（如时间）的限制下是否可解或可验证。这里只介绍这四个问题的简单定义。

P 问题、NP 问题、NPC 问题和 NPH 问题之间的关系如图2–1所示。

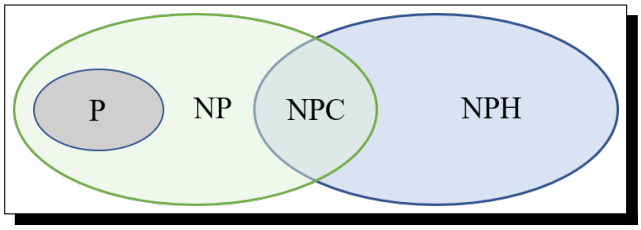

图 2–1 P 问题、NP 问题、NPH 问题和 NPC 问题之间的关系示意

2.3.1 P 问题

P（Polynominal Time，多项式时间）问题是指存在多项式时间算法的问题，或者说存在确定性算法能快速求出确切解的问题。这类问题通常被认为是“容易”或“可高效解决”的。如果一个问题可以在多项式（如 $O(n^k)$）的时间复杂度内解决，即可以在多项式时间内找到该问题的一个解。例如，对一组数字进行排序，使用快速排序算法平均可以在 $O(n \log n)$ 时间内完成。

之所以要研究一个问题是否存在多项式时间算法，是因为研究一个时间复杂度比多项式算法还要复杂的算法是没有实际意义的。

2.3.2 NP 问题

NP（Non-deterministic Polynominal Time，非确定性多项式时间）问题是指其解的正确性可以在多项式时间内被验证的问题，即解的正确性可以快速验证，但未必能快速求出解。换句话说，如果你能猜出一个问题的候选解，你可以在合理的时间内检查这个解是否正确。NP 问题的关键点在于，我们尚未知道是否存在针对它们的多项式时间确定性算法（否则就是 P 问题了），但是可以在多项式时间内“猜”出该问题的一个解，因此是可以验证的。

显然，P 问题是 NP 问题的子集，因为若存在多项式时间算法的问题，总能在多项式时间内验证它。NP 问题的一个经典例子是“旅行商问题”：给定一个城市的地图，找出一条访问每个城市一次并回到起点的最短路径。验证一条给定路径是否满足条件很容易（检查是否每个城市都只访问一次且总长度符合要求），但要想找出这条最短路径，目前已知最好的算法在最坏情况下需要指数时间。

2.3.3 NPH 问题

NPH（NP-Hard，NP 困难）问题是指这样一个问题：任意 NP 问题都可以在多项式时间内归约为该问题，但该问题本身不一定是 NP 问题（即该问题的解可能无法在多项式时间内被验证）。“归约”的意思是：为了解决问题 A，先将问题 A 转化为另一个问题 B（显然，问题 B 会比问题 A 更复杂），解决问题 B 同时也间接解决了问题 A。例如，停机问题（判断一个程序是否会结束运行）就是不可判定的，它属于 NPH 问题，但不属于 NP 问题。

相比于下面要介绍的 NPC 问题，由于放宽了限定条件，因此 NPH 问题比 NPC 问题的范围要广，它很可能比所有的 NPC 问题的时间复杂度更高，从而更难以解决。

2.3.4 NPC 问题

NPC（NP-Complete，NP 完全）问题是 NP 问题与 NPH 问题的交集。它们是 NP 问题中最困难、最具代表性的问题。具体来说，NPC 问题是指这样一个 NP 问题：所有的 NP 问题都可以在多项式时间内规约成它。其定义要满足两个条件：首先，该问题得是一个 NP 问题；然后，所有的 NP 问题都可以规约到它。

因此，NPC 问题是 NP 问题的“核心”。如果针对任何一个 NPC 问题找到了多项式时间算法，相当于证明了 NP = P，即证明了：任何在多项式时间内可验证的问题，都是多项式时间内可解的问题。这意味着，如果有一天我们找到了一个解决 NPC 问题的多项式时间算法，那么所有 NP 问题都将在多项式时间内可解。反之，如果证明了任何

一个 NPC 问题不存在多项式时间算法，那么 P ≠ NP 就成立。旅行商问题、背包问题、布尔可满足性问题等都是著名的 NPC 问题。

注解 2.1 在计算复杂性理论中，背包问题是一个经典的组合优化问题，也是 NP 完全问题的代表之一。该问题在资源分配、投资组合、项目选择等实际场景中有广泛应用。

背包问题的核心描述是：给定一组物品，每个物品有各自的重量和价值，在不超过背包最大承重限制的前提下，如何选择物品装入背包，使得背包内物品的总价值最大。

该问题主要分为两类：

1. 0-1 背包问题：每个物品要么被完整放入（1），要么不放入（0），不能分割。这是最经典的形式，已被证明是 NP 完全问题。
2. 分数背包问题：允许将物品分割后放入一部分。这个问题可以用贪心算法在多项式时间内求得最优解，属于 P 问题。

背包问题在计算复杂性理论中的重要性在于：0-1 背包问题的 NP 完全性证明了它在计算上是“困难”的，不存在（在 P ≠ NP 的假设下）能在多项式时间内解决所有实例的通用算法。因此，它常被用作基准来衡量其他优化问题的难度，并催生了动态规划、近似算法等多种算法设计思想。

注解 2.2 布尔可满足性问题（Boolean Satisfiability Problem，简称 SAT）是计算复杂性理论中的一个经典问题：给定一个由布尔变量（真/假）、逻辑运算符（如与、或、非）构成的逻辑表达式，判断是否存在一组对这些变量的赋值，使得整个表达式的值为真。

该问题是第一个被证明的 NP 完全问题，这意味着它是所有 NP 问题中最具代表性的难题之一，在理论计算机科学、硬件验证、人工智能规划、软件测试等领域有广泛应用基础。

2.3.5 理论意义与实际应用

P 问题、NP 问题、NPH 问题和 NPC 问题这些概念不仅仅是理论上的空想，它们有着深刻的实际意义：

- **指导算法设计**：当遇到一个 NPH 问题时，工程师们明白寻求精确的最优解在计算上可能是不可行的。因此，他们会转向寻找近似算法、启发式算法或随机算法，以便在合理时间内得到一个“足够好”的解。
- **密码学的基石**：现代密码学（如 RSA 加密算法）的安全性很大程度上建立在“P ≠ NP”的假设之上。加密和解密（对于合法用户）是 P 问题，而破解密码则被设计成 NP 问题（甚至是 NPH 问题）。如果 NP = P，意味着存在一种快速破解这些密码的方法，那么当前许多加密体系将被颠覆。

- **千禧年难题**："P 是否等于 NP"是克莱数学研究所提出的七大千禧年大奖难题之一，解决它可获得百万美元奖金，其结论将对计算机科学和数学产生深远影响。

计算复杂性理论在 MPC 中的应用场景主要包括：为 MPC 协议提供形式化的安全定义；指导基于各种计算困难性假设（如离散对数、格问题等）的密码学原语构造；分析和优化 MPC 协议的计算与通信复杂度以实现高效实现；以及界定 MPC 的能力边界（例如，确定哪些函数类可以在恶意敌手模型下被安全计算）。

2.4 经典教材与奠基性论文

计算复杂性理论领域有许多经典的参考文献，它们从不同角度构建了该学科的理论体系。以下整理了经典教材与奠基性论文的清单，供深入学习参考。

2.4.1 经典教材

计算复杂性理论领域的经典教材包括：

- 《Computational Complexity: A Modern Approach》(Sanjeev Arora & Boaz Barak)：内容非常全面，不仅覆盖了 P、NP 等经典理论，还深入探讨了电路复杂性、去随机化、PCP 定理等现代前沿话题，适合作为进阶学习的核心参考书。
- 《Computational Complexity》(Christos Papadimitriou)：侧重于算法和逻辑的视角，对 NP 完全性、空间复杂性等核心概念有非常深刻的阐述。
- 《Introduction to the Theory of Computation》(Michael Sipser)：非常适合初学者，该书中将计算理论分为自动机、可计算性和复杂性三部分，讲解极其直观易懂，能帮读者快速建立对 P、NP 等核心问题的直观理解。
- 《Computers and Intractability: A Guide to the Theory of NP-Completeness》(Garey & Johnson)：一本工具书，包含了大量 NP 完全问题的证明和归约方法。虽然出版时间较早，但至今仍是研究算法难解性不可或缺的经典。

注解 2.3 PCP（Probabilistically Checkable Proofs，通常译为"概率可检查证明"或"概率可验证证明"）定理是计算复杂性理论中的一个里程碑式的结论，它揭示了 NP 问题可以用概率可验证的证明系统来刻画。

简单来说，该定理证明了一个惊人的事实：对于任何 NP 问题，都存在一种极其高效的"概率验证"方案：

- 验证者只需随机检查证明字符串中的常数个比特。
- 即使证明字符串存在微小错误，验证者也能以高概率发现。

一个直观比喻是：想象你要验证一本 1000 页的数学证明书是否正确。传统方法需要通读全书。而 PCP 定理告诉你，存在一种神奇的“编码”方式，能把这本书转换成另一本更厚的书。你只需要随机翻看这本厚书中的 3 到 5 个字符，就能以 99.99% 的把握判断原书证明是否正确。这种编码的存在性本身就是一个深刻而强大的数学结论。

该定理的核心意义与影响如下：

1. 对 NP 类的新刻画：它给出了 NP 类的一个等价定义，即 $\mathrm{NP} = \mathrm{PCP}[O(\log n), O(1)]$。这意味着，任何 NP 证明都可以被转化为一种格式，使得验证者只需随机检查其中极少的部分，就能确信证明的正确性。
2. 近似算法的硬度（hardness of approximation，专业术语，特指计算复杂性理论中，对于优化问题，其近似算法所能达到的最好性能比存在不可突破的计算极限）：PCP 定理是证明许多近似算法不可近似性的基石。例如，它直接导致了“最大可满足性问题（MAX-SAT）不存在某种性能比的近似算法，除非 P = NP”这类强硬度结果。
3. 与传统证明的对比：传统 NP 证明需要验证者阅读整个证明才能确信。而 PCP 定理表明，通过引入极小的随机性和允许极小的错误概率，验证成本可以降至极低。

该定理的证明非常复杂，融合了代数、概率和编码理论的深刻思想，是理论计算机科学最杰出的成果之一。

2.4.2 奠基性论文

计算复杂性理论领域的奠基性论文包括：

- Hartmanis J, Stearns R E. On the computational complexity of algorithms[J]. Transactions of the American Mathematical Society, 1965, 117: 285-306.
  开创了计算复杂性理论，首次系统地定义了时间复杂性和空间复杂性的概念，并证明了时间层次定理，奠定了该学科的基础。
- Cook S A. The complexity of theorem-proving procedures (1971)[J]. 2021.
  提出了 NP 完全性理论，证明了 SAT 问题（可满足性问题）是 NP 完全的，并提出了著名的 P vs. NP 问题，这是该领域最核心的里程碑之一。
- Karp R M. Reducibility among combinatorial problems[M]//50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009: 219-241.
  扩展了 NP 完全性理论，证明了 21 个经典组合问题（如旅行商问题、图着色问题等）

都是 NP 完全的，极大地丰富了 NP 完全问题的例子库，使该理论从抽象走向实用。

注解 2.4 在计算复杂性理论中，图着色问题是一个经典的 NPC 问题。它主要研究：给定一个图（由顶点和边组成）和一组颜色，是否能够为每个顶点分配一种颜色，使得任何两个由边直接相连的顶点颜色都不同。

该问题的核心变体是图的三着色问题，即判断一个图是否能用三种颜色完成上述着色。这个问题被证明是 NPC 的，这意味着目前没有已知的多项式时间算法能解决所有情况，并且它是衡量其他问题计算难度的“标尺”之一。

图着色问题在理论计算机科学中至关重要，是证明其他问题同样属于 NPC 类别的关键工具。